

1) What are two challenges with using K-Nearest-Neighbors on the MNIST dataset? If in doubt, you can implement it and see what happens.

One challenge with using K-Nearest-Neighbors on the MNIST dataset is that KNN is computationally expensive because it requires comparing each test image against all training images. Imagine if we had tens of thousands of samples, it would be very slow. Another challenge is the high dimensionality, which makes the Euclidean distance less meaningful.

2) Why does using a simple linear classifier work so well on MNIST?

Using a simple linear classifier works so well on MNIST because handwritten digits have distinct pixel patterns. For example, "1" is mostly vertical while "8" contains closed loops. With 784 features for each image, the data is high-dimensional which makes it easier to find a line that separates most digit pairs. As a result, many digit pairs are linearly separable so a simple linear classifier works well.

3) How can you extend the linear classifier method to the multi-class case?

One way to extend the linear classifier method to the multi-class case is the one-vs-one approach, where you train classifiers for each digit pair and then use a voting scheme to pick the final class based on which digit wins the most comparisons. Another option is the one-vs-all approach, where you train 10 classifiers (each digit against all others) and then assign the digit whose classifier gives the strongest score. Both strategies allow a binary linear classifier to be used for multi-class problems.